

Тема 5.1 Развертывание фреймворка Django на персональном компьютере и начало работы

Что такое Django

Django — это фреймворк для создания веб-приложений с помощью языка программирования Python.

Django был создан в 2005 году, когда веб-разработчики из газеты Lawrence Journal-World стали использовать Python в качестве языка для создания веб-сайтов. А в 2008 году вышел публичный первый релиз фреймворка. На сегодняшний день он продолжает развиваться. Так, текущей версией фреймворка на момент написания этой статьи является версия 6.0, которая вышла в декабре 2025 года. Но кроме основной версии регулярно выходят подверсии, а также обновления и исправления в безопасности.

Django довольно популярен. Он используется на многих сайтах, в том числе таких, как Pinterest, PBS, Instagram, BitBucket, Washington Times, Mozilla и многих других.

Фреймворк является бесплатным. Он развивается как open source, его исходный код открыт, его можно найти в репозитории на githube.

На Django можно создавать широкий диапазон веб-приложений: от небольших персональных сайтов до высоконагруженных сложных веб-сервисов.

Django по умолчанию предлагает готовую функциональность для ряда распространенных задач, например, систему аутентификации, генерацию карт сайта и т.д., благодаря чему нам можно не изобретать велосипед и достаточно взять уже готовые компоненты.

В Django большое внимание уделяется безопасности, благодаря чему фреймворк помогает разработчикам избежать многих распространенных проблем в системе безопасности, например, sql-инъекций.

Фреймворк Django реализует архитектурный паттерн Model-View-Template или сокращенно MVT, который по факту является модификацией распространенного в веб-программировании паттерна MVC (Model-View-Controller).

Схематично мы можем представить архитектуру MVT в Django следующим образом (рисунок 1):

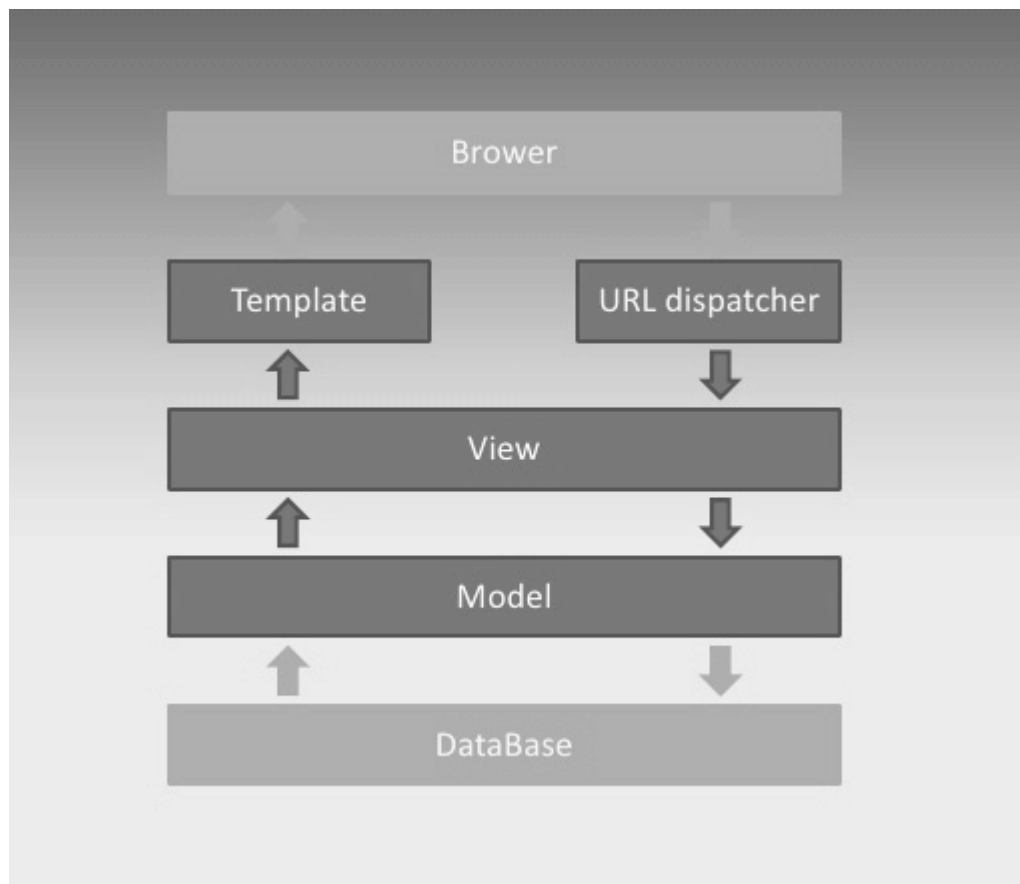


Рисунок 1 - Архитектура MVT в Django

Основные элементы паттерна:

- URL dispatcher: при получении запроса на основании запрошенного адреса URL определяет, какой ресурс должен обрабатывать данный запрос;
- View: получает запрос, обрабатывает его и отправляет в ответ пользователю некоторый ответ. Если для обработки запроса необходимо обращение к модели и базе данных, то View взаимодействует с ними. Для создания ответа может применять Template или шаблоны. В архитектуре MVC этому компоненту соответствуют контроллеры (но не представления);
- Model: описывает данные, используемые в приложении. Отдельные классы, как правило, соответствуют таблицам в базе данных;
- Template: представляет логику представления в виде сгенерированной разметки html. В MVC этому компоненту соответствует View, то есть представления.

Когда к приложению приходит запрос, то URL dispatcher определяет, с каким ресурсом сопоставляется данный запрос и передает этот запрос выбранному ресурсу. Ресурс фактически представляет функцию или View, который получает запрос и определенным образом обрабатывает его. В процессе обработки View может обращаться к моделям и базе данных, получать из нее данные, или, наоборот, сохранять в нее данные. Результат обработки запроса отправляется обратно, и этот результат пользователь видит в своем браузере. Как правило,

результат обработки запроса представляет сгенерированный html-код, для генерации которого применяются шаблоны (Template).

Установка и настройка Django

Существуют разные способы установки Django. Рассмотрим рекомендуемый способ.

Пакеты Django размещаются в центральном репозитории для большинства пакетов Python - Package Index (PyPI). И для установки из этого репозитория нам потребуется пакетный менеджер pip. Менеджер pip позволяет загружать пакеты и управлять ими. Обычно при установке python также устанавливается и менеджер pip. В этом случае мы можем проверить версию менеджера, выполнив в командной строке/терминале команду `pip -V` (V - с заглавной буквы):

```
C:\Users\eugen>pip -V
pip 23.3.1 from
C:\Users\eugen\AppData\Local\Programs\Python\Python312\Lib\site-packages\pip
(python 3.12)
```

```
C:\Users\eugen>
```

Если pip не установлен, то мы увидим ошибку типа "pip" не является внутренней или внешней командой, исполняемой программой или пакетным файлом

В этом случае нам надо установить pip. Для этого можно выполнить в командной строке/консоли следующую команду:

```
python3 -m ensurepip --upgrade
```

Если pip ранее уже был установлен, то можно его обновить с помощью команды

```
python3 -m pip install --upgrade pip
```

Виртуальная среда или venv не является неотъемлемой частью разработки на Django. Однако ее рекомендуется использовать, так как она позволяет создать множество виртуальных сред Python на одной операционной системе. Благодаря виртуальной среде приложение может запускаться независимо от других приложений на Python.

В принципе можно запускать приложения на Django и без виртуальной среды. В этом случае все пакеты Django устанавливаются глобально. Однако что если после создания первого приложения выйдет новая версия Django? Если мы захотим использовать для второго проекта новую версию Django, то из-за глобальной установки пакетов придется обновлять первый проект, который использует старую версию. Это потребует некоторой дополнительной работы по обновлению, так как не всегда соблюдается обратная совместимость между пакетами. Если мы решим использовать для второго проекта старую версию, то мы лишимся потенциальных преимуществ новой версии. И использование виртуальной среды как раз позволяет разграничить пакеты для каждого проекта.

Для работы с виртуальной средой в python применяется встроенный модуль venv.

Итак, создадим виртуальную среду. Вначале определим каталог для проектов django. Например, пусть это будет каталог C:\django. Прежде всего перейдем в терминале/командной строке в этот каталог с помощью команды cd.

```
cd C:\django
```

Затем для создания виртуальной среды выполним следующую команду:

```
python3 -m venv .venv
```

Модулю venv передается название среды, которая в данном случае будет называться ".venv". Для наименования виртуальных сред нет каких-то определенных условностей. Пример консольного вывода:

```
C:\Users\eugen>cd C:\djangoПереход к папке будущей виртуальной среды
```

```
C:\django>python3 -m venv .venvСоздание виртуальной среды
```

```
C:\django>
```

После этого в текущей папке (C:\django) будет создан подкаталог ".venv".

Для использования виртуальную среду надо активировать. И каждый раз, когда мы будем работать с проектом Django, связанную с ним виртуальную среду надо активировать. Например, активируем выше созданную среду, которая располагается в текущем каталоге в папке .venv. Процесс активации немного отличается в зависимости от операционной системы и от того, какие инструменты применяются. Так, в Windows можно использовать командную строку и PowerShell, но между ними есть отличия.

Если наша ОС - Windows, то в папке .venv/Scripts/ мы можем найти файл activate.bat), который активирует виртуальную среду. Так, в Windows активация виртуальной среды в командной строке будет выглядеть таким образом:

```
.venv\Scripts\activate.bat
```

Также при работе на Windows в папке .venv/Scripts/ мы можем найти файл activate.ps1, который также активирует виртуальную среду, но применяется только в PowerShell. Но при работе с PowerShell следует учитывать, что по умолчанию в этой оболочке запрещено применять скрипты. Поэтому перед активацией среды необходимо установить разрешения для текущего пользователя. Поэтому для активации виртуальной среды в PowerShell необходимо выполнить две следующих команды:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

```
.venv\Scripts\Activate.ps1
```

Для Linux и MacOS активация будет производиться с помощью следующей команды:

```
source .venv/bin/activate
```

Далее я буду приводить примеры на основе командной строки Windows, однако все остальные примеры не будут зависеть от того, что используется - PowerShell или командная строка, Windows, Linux или MacOS. В любом случае после успешной активации слева от текущего каталога мы увидим в скобках название виртуальной среды:

```
C:\Users\eugen>cd C:\django
```

```
C:\django>python3 -m venv .venv
```

```
C:\django>.venv\Scripts\activate.batАктивация виртуальной среды
```

```
(.venv) C:\django> Виртуальная среда активирована
```

После активации виртуальной среды для установки Django выполним в консоли следующую команду

```
python3 -m pip install Django
```

Она устанавливает последнюю версию Django.

```
(.venv) C:\django>python -m pip install Django
```

```
Collecting Django
```

```
  Downloading django-6.0-py3-none-any.whl.metadata (3.9 kB)
```

```
Collecting asgiref>=3.9.1 (from Django)
```

```
  Downloading asgiref-3.11.0-py3-none-any.whl.metadata (9.3 kB)
```

```
Collecting sqlparse>=0.5.0 (from Django)
```

```
  Downloading sqlparse-0.5.4-py3-none-any.whl.metadata (4.7 kB)
```

```
Downloading django-6.0-py3-none-any.whl (8.3 MB)
```

```
8.3/8.3 MB 1.5 MB/s eta 0:00:00
```

```
Downloading asgiref-3.11.0-py3-none-any.whl (24 kB)
```

```
Downloading sqlparse-0.5.4-py3-none-any.whl (45 kB)
```

```
45.9/45.9 kB 2.9 MB/s eta 0:00:00
```

```
Installing collected packages: sqlparse, asgiref, Django
```

```
Successfully installed Django-6.0 asgiref-3.11.0 sqlparse-0.5.4
```

```
(.venv) C:\django>
```

Если нам интересуют конкретная версия Django, то мы можем указать ее при установке:

```
python3 -m pip install django~=5.0.0
```

Чтобы убедиться, что все установлено правильно, мы можем перейти к интерпретатору python. Для этого введем в терминале команду

```
python3
```

И затем выполним последовательно следующие две инструкции:

```
>>> import django
```

```
>>> print(django.get_version())
```

Консольный вывод в моем случае:

```
(.venv) C:\django>python3
```

```
Python 3.12.3 (main, Nov 6 2025, 13:44:16) [GCC 13.3.0] on linux
```

```
Type "help", "copyright", "credits" or "license" for more information.
```

```
>>> import django
```

```
>>> print(django.get_version())
```

```
6.0
```

```
>>>
```

После окончания работы с виртуальной средой мы можем ее деактивировать с помощью команды:

```
Deactivate
```

Создание первого проекта

При установке Django в папке виртуальной среды устанавливается утилита django-admin. А на Windows также исполняемый файл django-admin.exe. Их можно найти в папке виртуальной среды, в которую производилась установка Django: на Windows - в подкаталоге Scripts, а на Linux/MacOS - в каталоге bin (рисунок 2).

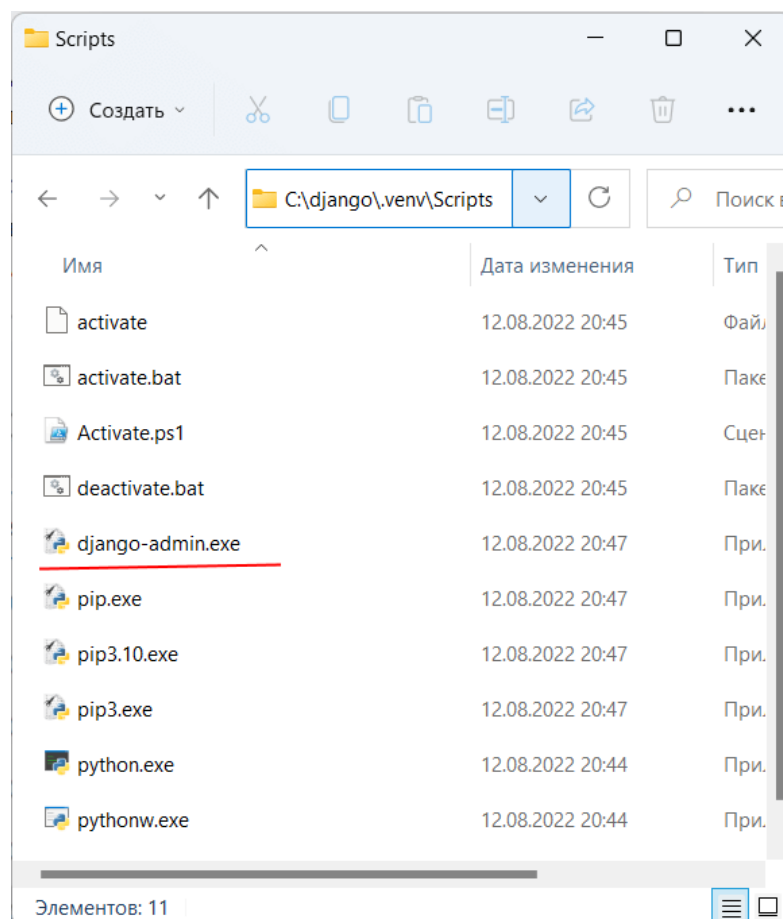


Рисунок 2 - Django-admin в Django и Python

django-admin предоставляет ряд команд для управления проектом Django. В частности, для создания проекта применяется команда startproject. Этой команде в качестве аргумента передается название проекта.

Итак, создадим первый на Django. Пусть он будет располагаться в той же папке, где располагается каталог виртуальной среды. И для этого вначале активируем ранее созданную виртуальную среду (например, среду .venv, которая была создана в прошлой теме, если она ранее не была активирована) (рисунок 3).

И после активации виртуальной среды выполним следующую команду
c:\django>django-admin startproject metanit

```

Администратор: Командная
Microsoft Windows [Version 10.0.22000.856]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

C:\Users\eugen>cd c:\django      Переход к папке будущего проекта

c:\django>.venv\Scripts\activate  Активация виртуальной среды

(.venv) c:\django>django-admin startproject metanit  Создание проекта

(.venv) c:\django>

```

Рисунок 3 - Создание проекта на Python и Django

В данном случае мы создаем проект с именем "metanit". И после выполнения этой команды в текущей папке (c:\django) будет создан каталог metanit (рисунок 4).

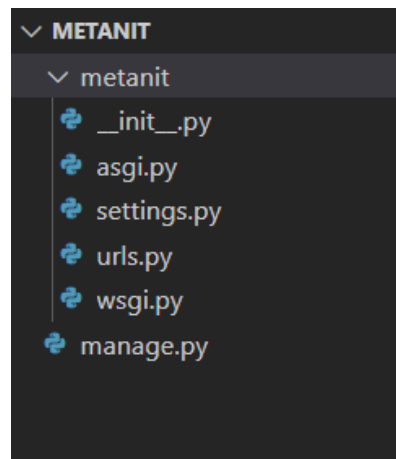


Рисунок 4 - Первый проект на Python и Django

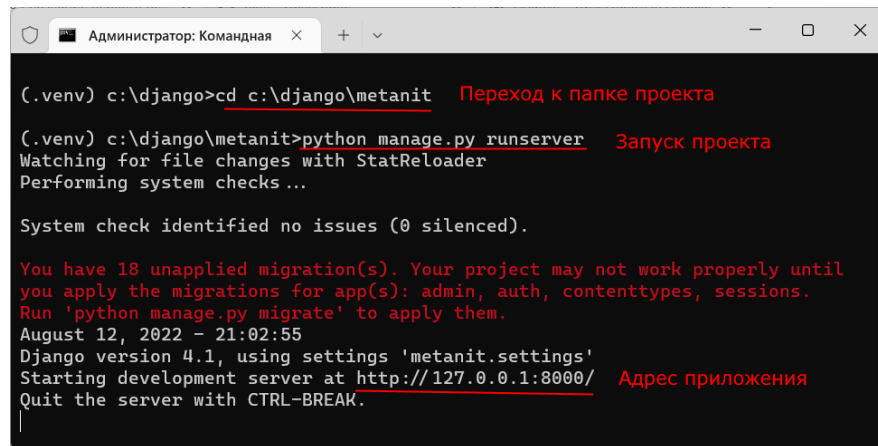
Созданный каталог будет состоять из следующих элементов:

- manage.py: выполняет различные команды проекта, например, создает и запускает приложение
- metanit - собственно папка проекта metanit, которая содержит следующие файлы:
 - __init__.py: данный файл указывает, что папка, в которой он находится, будет рассматриваться как модуль. Это стандартный файл для программы на языке Python;
 - settings.py: содержит настройки конфигурации проекта;
 - urls.py: содержит шаблоны URL-адресов, по сути определяет систему маршрутизации проекта;
 - wsgi.py: содержит свойства конфигурации WSGI (Web Server Gateway Interface). Он используется при развертывании проекта;

– asgi.py: название файла представляет сокращение от Asynchronous Server Gateway Interface и расширяет возможности WSGI, добавляя поддержку для взаимодействия между асинхронными веб-серверами и приложениями.

Запустим проект на выполнение. Для этого с помощью команды `cd` перейдем в консоли к папке проекта (рисунок 5). И затем для запуска проекта выполним следующую команду:

```
python manage.py runserver
```



```
(.venv) c:\django>cd c:\django\metanit
(.venv) c:\django\metanit>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks ...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly until
you apply the migrations for app(s): admin, auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.
August 12, 2022 - 21:02:55
Django version 4.1, using settings 'metanit.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Рисунок 5 - Запуск первого проекта на Python и Django

После запуска проекта в консоли мы увидим адрес, по которому запущен проект. Как правило, это адрес `http://127.0.0.1:8000/`. Откроем любой веб-браузер и введем данный адрес в адресную строку браузера. И нам откроется содержимое по умолчанию (рисунок 6).

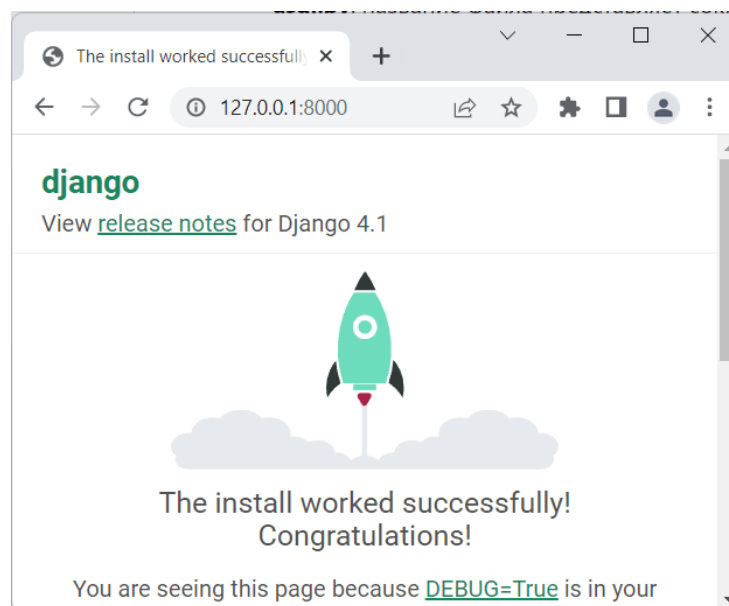


Рисунок 6 - Первое веб-приложение на Python и Django

Структура Django

Фреймворк основывается на четырёх главных компонентах:

- модели (Models) — взаимодействуют с базой данных и достают из неё ту информацию, которую необходимо отобразить в браузере;
- представления (Views) — обрабатывают запрос и обращаются к модели, сообщая ей, какую информацию необходимо достать из базы данных;
- шаблоны (Templates) — показывают, каким именно образом необходимо показать информацию, полученную из базы данных;
- URL-маршруты (URL dispatcher) — перенаправляют HTTP-запрос от браузера в представления.

Практическая часть

1 Установка Visual Studio Code Зайдите на сайт <https://code.visualstudio.com/> и скачайте Visual Studio Code;

2 Установка Python Extension Pack:

Заходим в программу VS Code. Слева ищем иконку Расширения (Extensions) и нажимаем. Появляется поисковая строка в ней пишем Python Extension Pack и скачиваем расширение (рисунок 7).

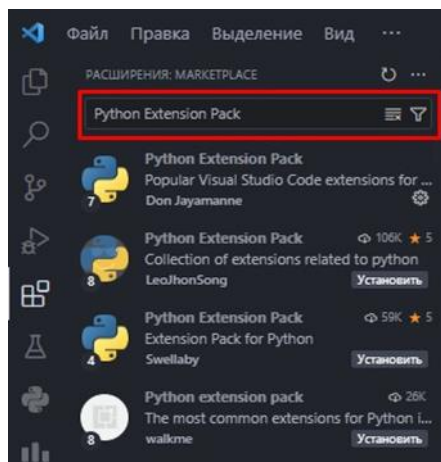


Рисунок 7 - Расширение Python Extension

3 Установка виртуального окружения и Django:

Создайте новую папку на локальном диске и назовите её под вашей Фамилией. В ней создайте папку и назовите её

«myproject». Это будет вашей папкой проекта! В эту папку мы и будем устанавливать наше виртуальное окружение. После создания папок, откроем в VS Code папку нашего проекта. Если вы все сделали правильно, у вас будет открыта папка проекта (рисунок 8).

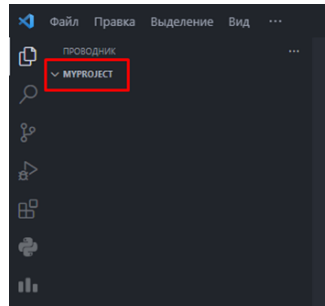


Рисунок 8 - Папка проекта в VS Code

Открываем терминал. Его можно открыть двумя способами:

- 1 В командном центре сверху найти вкладку «Терминал» и нажать на кнопку «Создать терминал» (рисунок 9);

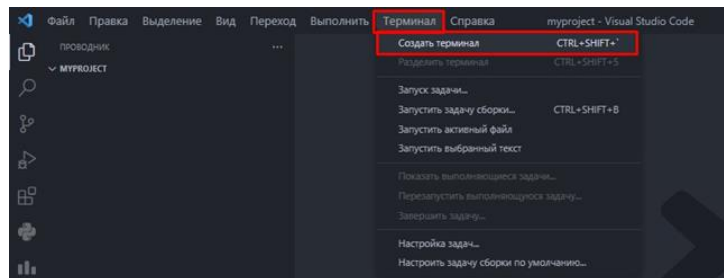


Рисунок 9 - Команда «Создать терминал» в командном центре в VS Code

- 2 Нажать сочетание клавиш «Ctrl + Shift + `» (` – буква «ё»)

Пишем поочередно список из следующих команд, чтобы создать виртуальное окружение и установить Django (рисунок 10).

```
PS D:\django-learning\myproject> python -m venv venv
```

Рисунок 10 - Создание виртуального окружения

- У нас появилась папка виртуального окружения в папке проекта (рисунок 11).

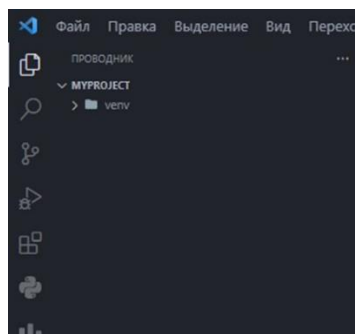
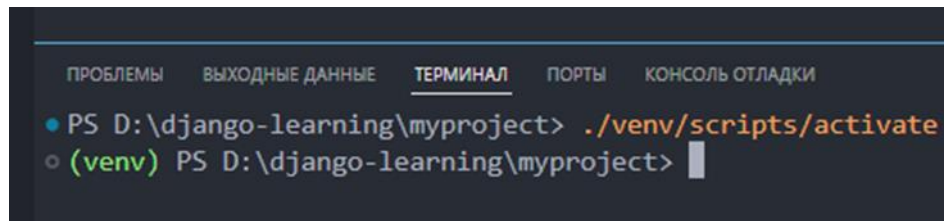


Рисунок 11 - Папка виртуального окружения «venv»

Далее, необходимо активировать виртуальное окружение (рисунок 12).



```

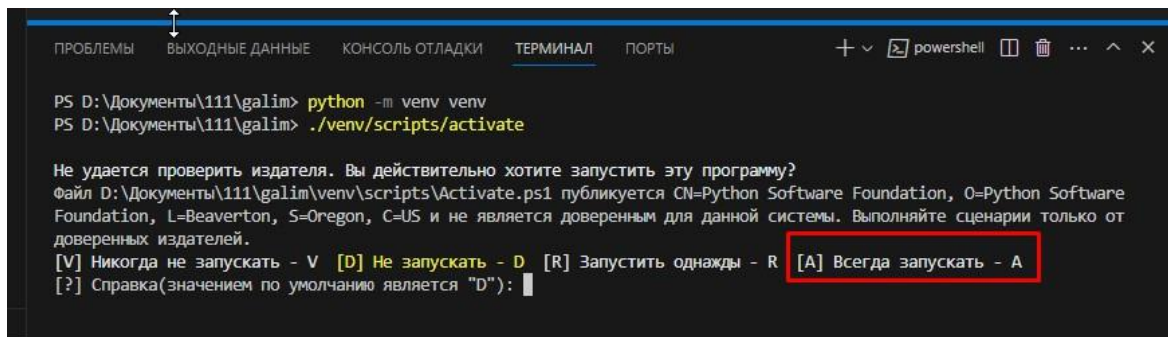
ПРОБЛЕМЫ  ВЫХОДНЫЕ ДАННЫЕ  ТЕРМИНАЛ  ПОРТЫ  КОНСОЛЬ ОТЛАДКИ

• PS D:\django-learning\myproject> ./venv/scripts/activate
○ (venv) PS D:\django-learning\myproject>

```

Рисунок 12 - Активация виртуального окружения

Если в терминале появляется предупреждение об отсутствии возможности проверки издателя, и предлагает запустить программу отправляем английскую букву А (рисунок 13).



```

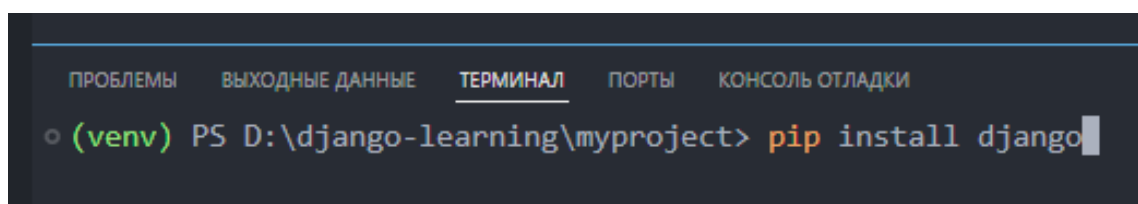
ПРОБЛЕМЫ  ВЫХОДНЫЕ ДАННЫЕ  КОНСОЛЬ ОТЛАДКИ  ТЕРМИНАЛ  ПОРТЫ  powershell  ...  ^  X

PS D:\Документы\111\galim> python -m venv venv
PS D:\Документы\111\galim> ./venv/scripts/activate

Не удастся проверить издателя. Вы действительно хотите запустить эту программу?
Файл D:\Документы\111\galim\venv\scripts\Activate.ps1 публикуется CN=Python Software Foundation, O=Python Software
Foundation, L=Beaverton, S=Oregon, C=US и не является доверенным для данной системы. Выполняйте сценарии только от
доверенных издателей.
[V] Никогда не запускать - V [D] Не запускать - D [R] Запустить однажды - R [A] Всегда запускать - A
[?] Справка(значением по умолчанию является "D"):
```

Рисунок 13 - Предупреждение об отсутствии возможности проверки издателя

3 Скачаем Django в папку проекта и немного подождем, пока Django установится (рисунок 14).



```

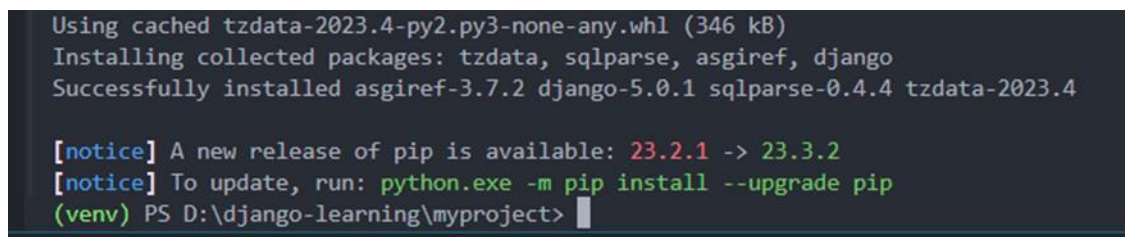
ПРОБЛЕМЫ  ВЫХОДНЫЕ ДАННЫЕ  ТЕРМИНАЛ  ПОРТЫ  КОНСОЛЬ ОТЛАДКИ

○ (venv) PS D:\django-learning\myproject> pip install django

```

Рисунок 14 - Команда для установки Django

Поздравляю, Django установлен (рисунок 15)!



```

Using cached tzdata-2023.4-py2.py3-none-any.whl (346 kB)
Installing collected packages: tzdata, sqlparse, asgiref, django
Successfully installed asgiref-3.7.2 django-5.0.1 sqlparse-0.4.4 tzdata-2023.4

[notice] A new release of pip is available: 23.2.1 -> 23.3.2
[notice] To update, run: python.exe -m pip install --upgrade pip
(venv) PS D:\django-learning\myproject>

```

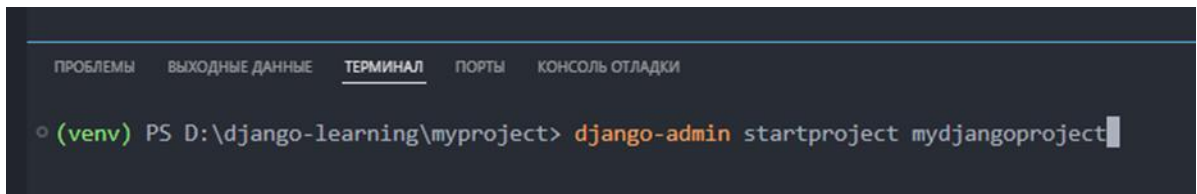
Рисунок 15 - Уведомление об успешной установке Django

Также, VS Code говорит нам, что наша версия `pip`'а немного устарела, для обновления пропишите команду. Следуем его рекомендациям и прописываем данную команду в терминале (рисунок 16).

```
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 23.2.1
    Uninstalling pip-23.2.1:
      Successfully uninstalled pip-23.2.1
    Successfully installed pip-23.3.2
(venv) PS D:\django-learning\myproject> █
```

Рисунок 16 - Уведомление об успешном обновлении пакетного менеджера `pip`

4 Создание нового Django-проекта (рисунок 17).



```
PROБЛЕМЫ  ВЫХОДНЫЕ ДАННЫЕ  ТЕРМИНАЛ  ПОРТЫ  КОНСОЛЬ ОТЛАДКИ
(venv) PS D:\django-learning\myproject> django-admin startproject mydjango project █
```

Рисунок 17 - Команда для создания проекта в Django

- **mydjango project** – имя проекта. После выполнения этой команды появится папка **Django-проекта mydjango project**, в которой будет папка с точно таким же названием, как и папка Django-проекта, со следующей структурой (рисунок 18):

- **manage.py** – управляющий скрипт, который используется для различных административных задач, таких как запуск сервера, создание миграций, управление командами Django и т.д.;

- **init.py** – пустой файл, который говорит Python'у, что mydjango project должен рассматриваться как пакет;

- **settings.py** – в этом файле хранятся настройки проекта, такие как БД, настройки безопасности, используемый шаблон и т.д.;

- **urls.py** – файл, который содержит ссылки (иными словами, URL-адреса или «маршруты») проекта, определяющие, какие представления должны быть вызваны для определенных URL-адресов;

- **wsgi.py** – файл используется для развертывания проекта на сервере, который поддерживает WSGI (Web Server Gateway Interface).

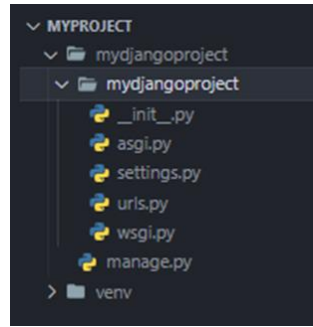


Рисунок 18 - Структура проекта

5 Запуск Django-проекта

Для запуска Django-проекта, перейдем в папку mydjango project следующей командой для смены директории (рисунок 19):

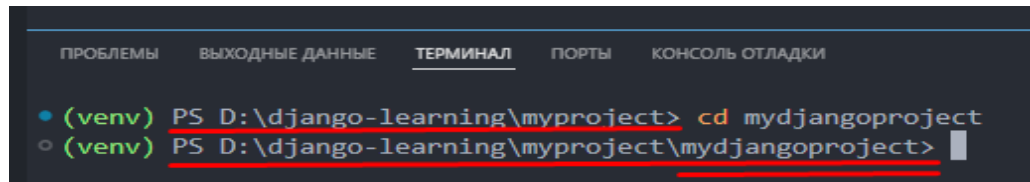


Рисунок 19 - Команда cd для смены директории на уровень выше

Далее, необходимо прописать команду для запуска Django-проекта на локальном сервере (рисунок 20).

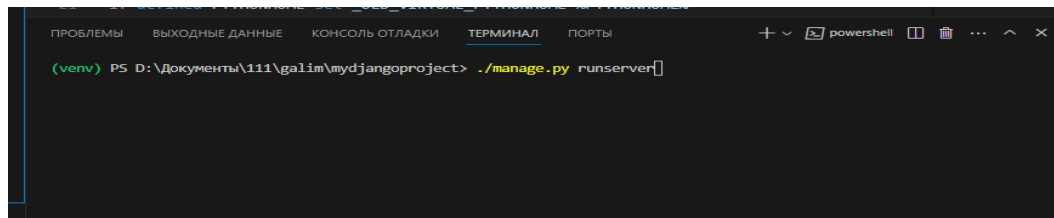


Рисунок 20 - Команда для запуска Django-проекта

Если команда ./manage.py runserver не работает, напишите полную версию команды (python manage.py runserver) (рисунок 21).

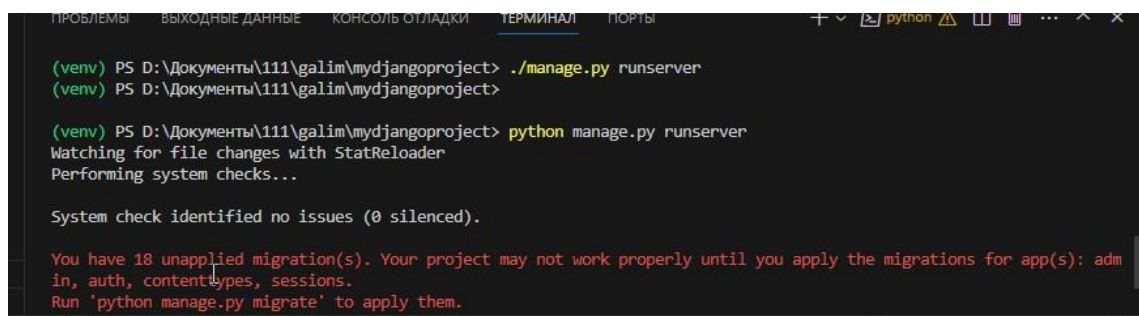
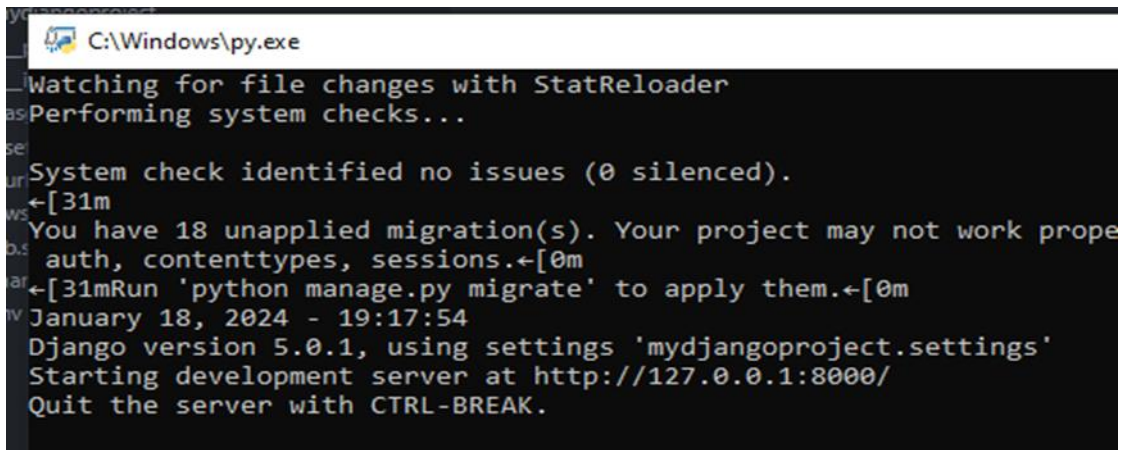


Рисунок 21 - Запуск сервера командой python manage.py runserver

Поздравляю! Мы запустили локальный сервер (рисунок 22).



```

C:\Windows\py.exe
Watching for file changes with StatReloader
Performing system checks...

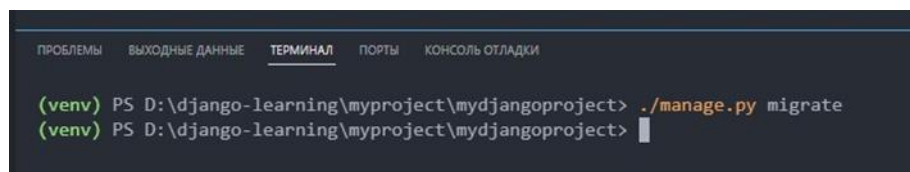
System check identified no issues (0 silenced).
<[31m
You have 18 unapplied migration(s). Your project may not work properly
auth, contenttypes, sessions.<[0m
<[31mRun 'python manage.py migrate' to apply them.<[0m
January 18, 2024 - 19:17:54
Django version 5.0.1, using settings 'mydjangoapp.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
  
```

Рисунок 22 - Запущенный локальный сервер по адресу <http://127.0.0.1:8000/>

Если Вы заметили, то в консоли красными буквами появилось предупреждение, что имеется 18 несозданных миграций.

Миграция – процесс автоматического обновления базы данных при изменении модели приложения.

Выполним миграции командой `./manage.py migrate` (рисунок 23).



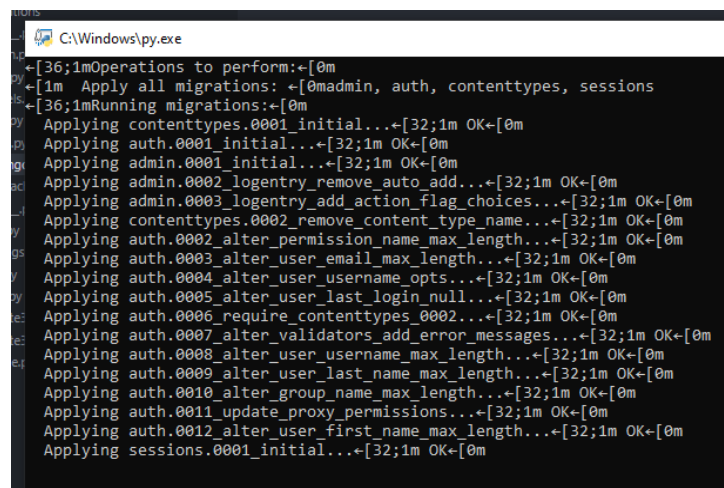
```

ПРОБЛЕМЫ  ВЫХОДНЫЕ ДАННЫЕ  ТЕРМИНАЛ  ПОРТЫ  КОНСОЛЬ ОТЛАДКИ

(venv) PS D:\django-learning\myproject\mydjangoapp> ./manage.py migrate
(venv) PS D:\django-learning\myproject\mydjangoapp>
  
```

Рисунок 23 - Выполнение миграций командой `./manage.py migrate`

После того, как мы пропишем данную команду и нажмем на Enter, то у нас начнут выполняться миграции (рисунок 24).



```

C:\Windows\py.exe
<[36;1mOperations to perform:<[0m
<[31m Apply all migrations: <[0madmin, auth, contenttypes, sessions
<[36;1mRunning migrations:<[0m
Applying contenttypes.0001_initial...<[32;1m OK<[0m
Applying auth.0001_initial...<[32;1m OK<[0m
Applying admin.0001_initial...<[32;1m OK<[0m
Applying admin.0002_logentry_remove_auto_add...<[32;1m OK<[0m
Applying admin.0003_logentry_add_action_flag_choices...<[32;1m OK<[0m
Applying contenttypes.0002_remove_content_type_name...<[32;1m OK<[0m
Applying auth.0002_alter_permission_name_max_length...<[32;1m OK<[0m
Applying auth.0003_alter_user_email_max_length...<[32;1m OK<[0m
Applying auth.0004_alter_user_username_opts...<[32;1m OK<[0m
Applying auth.0005_alter_user_last_login_null...<[32;1m OK<[0m
Applying auth.0006_require_contenttypes_0002...<[32;1m OK<[0m
Applying auth.0007_alter_validators_add_error_messages...<[32;1m OK<[0m
Applying auth.0008_alter_user_username_max_length...<[32;1m OK<[0m
Applying auth.0009_alter_user_last_name_max_length...<[32;1m OK<[0m
Applying auth.0010_alter_group_name_max_length...<[32;1m OK<[0m
Applying auth.0011_update_proxy_permissions...<[32;1m OK<[0m
Applying auth.0012_alter_user_first_name_max_length...<[32;1m OK<[0m
Applying sessions.0001_initial...<[32;1m OK<[0m
  
```

Рисунок 24 - Выполнение миграций

После выполнения миграций, снова запустим проект и перейдем по ссылке локального сервера (можно зажать Ctrl и нажать на ссылку) (рисунок 25).

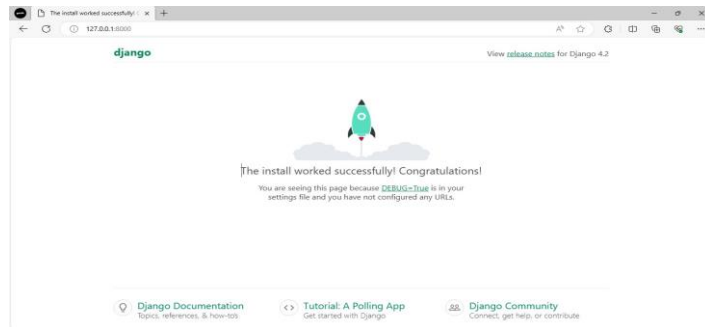


Рисунок 25 - Запущенный локальный сервер в браузере

6 Создание Django-приложения (Django-app)

Django-app – модуль, который выполняет конкретные функции в рамках веб-приложения. Каждое приложение включает в себя свою структуру URL-адресов, модели данных, представления (views), шаблоны и статические файлы. Django-app's логически разделяют функциональность веб-приложения на более управляемые компоненты (рисунок 26).

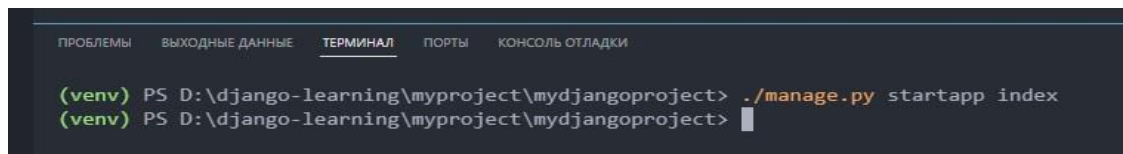


Рисунок 26 - Команда для создания Django-приложения

В папке Django-проекта мы видим изменение. Добавилась папка index. В ней мы видим кучу новых файлов, но обо всем по порядку (рисунок 27).

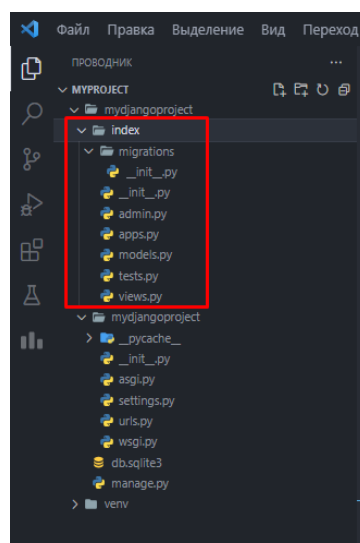


Рисунок 27 Папка Django-приложения (index)

Структура Django-приложения включает следующие основные элементы (таблица 1):

Таблица 1 - Структура приложения

Структура приложения	
Папка migrations	Создание файлов миграции
admin.py	Регистрация таблиц в БД для админов
apps.py	Информация о приложении
models.py	Создание таблиц для БД
tests.py	Создание тестов для проекта
views.py	Создание функций, для отображения шаблонов на сайте

7 Осуществим **Регистрацию приложения** в файле настроек проекта.

Заходим в файл settings.py и в списке констант INSTALLED_APPS указываем index, для того, что зарегистрировать наше веб-приложение (рисунок 28).

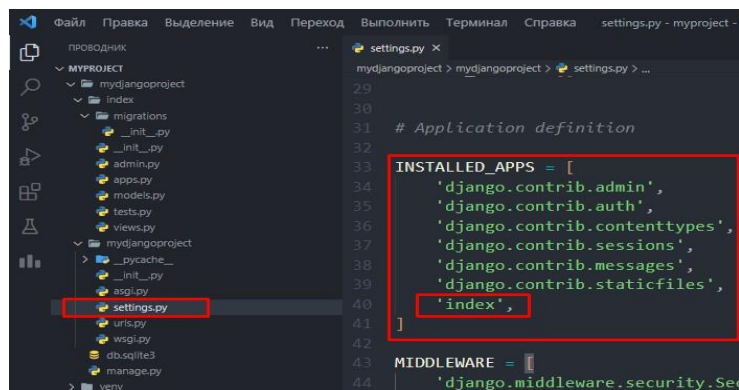


Рисунок 28 - Регистрация Django-приложения в INSTALLED_APPS

Создадим папку templates (файлы шаблонов в папке приложения)

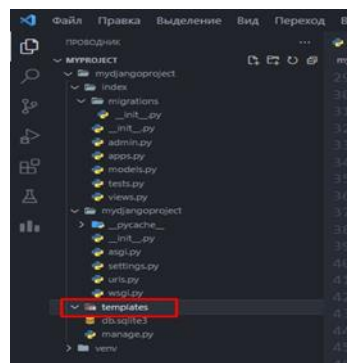
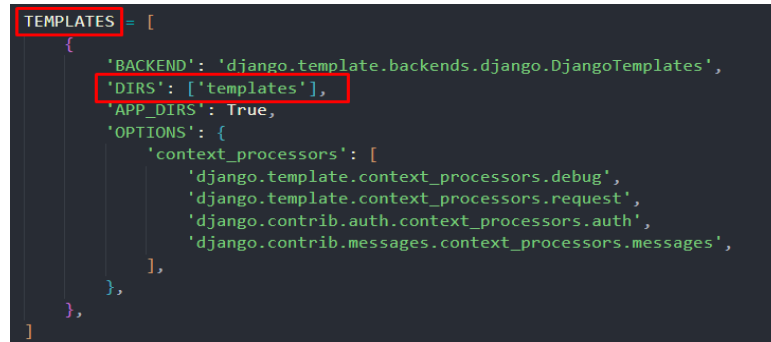


Рисунок 28 - Папка templates

После создания папки для хранения файлов шаблонов веб-приложения, её необходимо зарегистрировать в нашем проекте.

8 Регистрация папки templates

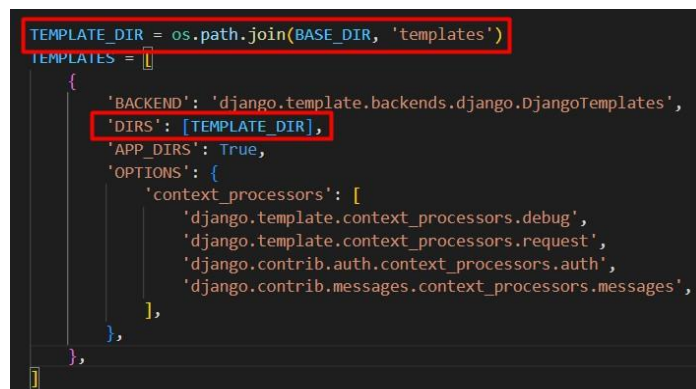
За конфигурацию шаблонов в проекте отвечает переменная `TEMPLATES` в файле `settings.py`. Укажем в «DIRS» в квадратных скобках (в списке) строку `templates` (рисунок 29).



```
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': ['templates'],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]
```

Рисунок 29 - Регистрация папки `templates` в `TEMPLATES`

Из рекомендаций: для регистрации папки `templates` лучше всего использовать константу директории `TEMPLATE_DIR` и передавать её в ключ «DIRS» (рисунок 30).



```
TEMPLATE_DIR = os.path.join(BASE_DIR, 'templates')
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [TEMPLATE_DIR],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]
```

Рисунок 30 - Создание константы директории `TEMPLATE_DIR`

9 Создание файла `index.html`.

Далее, создадим файл `index.html` в папке `templates` (рисунок 31).

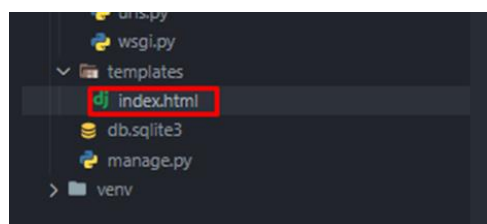
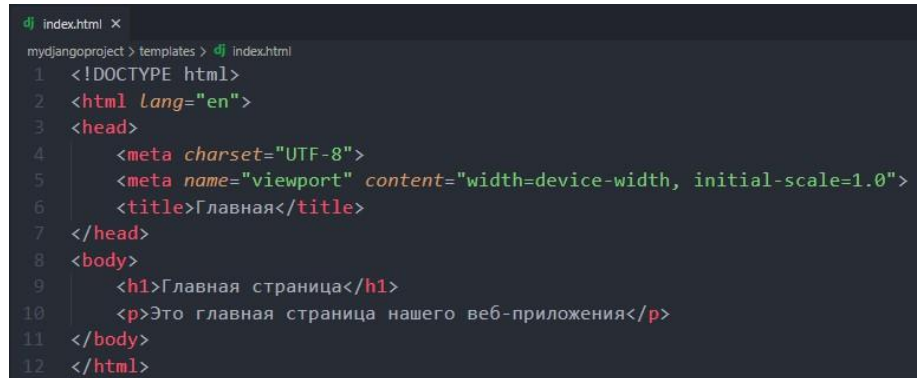


Рисунок 31 - Файл `index.html` в папке `templates`

Пропишем в данном файле небольшой код на языке гипертекстовой разметки HTML, который будет выводить в теге заголовка первого уровня текст «Главная страница» и в теге параграфа текст «Это главная страница нашего веб-приложения» (рисунок 32).



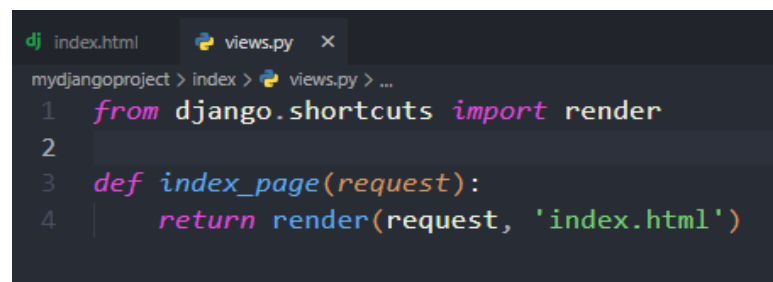
```

dj index.html X
mydjango project > templates > dj index.html
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Главная</title>
7 </head>
8 <body>
9   <h1>Главная страница</h1>
10  <p>Это главная страница нашего веб-приложения</p>
11 </body>
12 </html>

```

Рисунок 32 - Код для файла index.html

10 Создание функции в views.py для открытия главной страницы. Создадим функцию для открытия страницы в файле views.py (рисунок 33).



```

dj index.html views.py X
mydjango project > index > views.py > ...
1 from django.shortcuts import render
2
3 def index_page(request):
4     return render(request, 'index.html')

```

Рисунок 33 - Функция index_page для обработки страницы index.html

Здесь, немного остановимся и давайте поймем, что мы написали. Мы определяем функцию с именем **index_page**, которая принимает аргумент **request**. В Django **каждая функция представления** принимает объект **request**, представляющий запрос, отправленный клиентом (веб-браузером).

`return render(request, 'index.html')`

Эта строка означает, что при обращении к представлению **index_page**, мы хотим **вернуть HTML-страницу**, которая будет отображаться пользователю.

Функция **render** отвечает за обработку запроса и генерацию HTML-страницы на основе шаблона.

11 Определим маршрут в файле urls.py для созданной view-функции (рисунок 34).

```

17 from django.contrib import admin
18 from django.urls import path
19 from index.views import index_page
20
21 urlpatterns = [
22     path('admin/', admin.site.urls),
23     path('', index_page)
24 ]

```

Рисунок 34 - Определение маршрута для главной страницы

Остановимся на этом этапе и поймем, что мы сделали.

- **urlpatterns** – переменная, которая содержит список URL-адресов и соответствующих обработчиков представлений;
- **path('admin/', admin.site.urls)** – данный путь веб-сайта связан с админ панелью. Когда пользователь переходит по адресу, который начинается с '/admin/', Django будет направлять запрос на соответствующий обработчик для административной панели;
- **path('', index_page)** – данный путь веб-сайта связан с обработчиком представления `index_page`. Когда пользователь посещает корневой URL (например, `http://example.com/`), Django будет направлять запрос на функцию `index_page`;
- **from index.views import index_page** – из файла **views**, который находится в папке приложения **index** мы импортируем функцию **index_page**.

12 Запустим Django-проект (рисунок 35)!

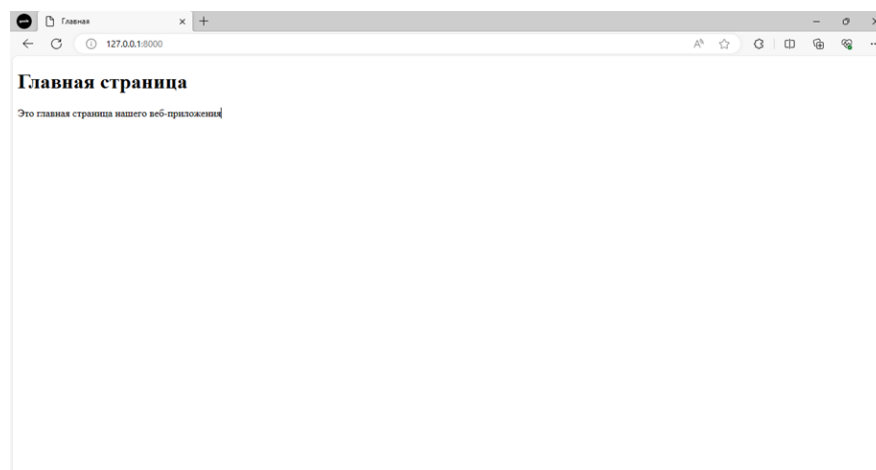


Рисунок 35 - Запущенный локальный сервер проекта

Пример решения задачи на языке программирования Python

Задача. Создайте веб-проект на фреймворке Django с именем **blog_site**. Внутри проекта создайте приложение **posts**, предназначенное для работы с

публикациями блога. Добавьте созданное приложение в список `INSTALLED_APPS` в файле `settings.py`. После этого запустите встроенный сервер разработки Django и убедитесь, что проект успешно запускается и открывается в браузере по адресу `http://127.0.0.1:8000`.

Решение задачи:

1. Создание проекта
`django-admin startproject blog_site`
`cd blog_site`
2. Создание приложения
`python manage.py startapp posts`
3. Подключение приложения

Откройте файл `blog_site/settings.py` и добавьте приложение в `INSTALLED_APPS`:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'posts', # добавленное приложение
]
```

4. Запуск сервера
`python manage.py runserver`
5. Проверка
 Откройте в браузере:
`http://127.0.0.1:8000/`

Использование моделей

Модель — единственный и окончательный источник информации о ваших данных. Он содержит основные поля и поведение хранимых вами данных. Обычно каждой модели соответствует одна таблица в базе данных.

Каждая модель — класс Python, который наследуется от `django.db.models.Model`.

Каждый атрибут модели представляет поле базы данных. Наряду со всем этим Django предоставляет API для доступа к автоматически сгенерированной базе данных.

После определения моделей Вам нужно сообщить Django, что вы хотите использовать эти модели. Вы сможете реализовать данный функционал, отредактировав файл настроек и изменив настройку, `INSTALLED_APPS` добавив имя модуля, который его содержит `models.py`

Например, если ваша модель приложения находится в модуле `myapp.models` (структура пакета, созданная для приложения скриптом), должна содержать:
`manage.py startapp INSTALLED_APPS`

Подключение к базе данных

Для хранения данных в веб-приложении, как правило, применяются базы данных. И фреймворк Django уже по умолчанию предоставляет удобный функционал для работы с различными системами баз данных.

По умолчанию Django в качестве базы данных использует SQLite. Она очень проста в использовании и не требует запущенного сервера. Все файлы базы данных могут легко переноситься с одного компьютера на другой. Однако при необходимости мы можем использовать в Django большинство распространенных СУБД.

Для работы с базами данных в проекте Django в файле settings.py определен параметр DATABASES, который по умолчанию выглядит следующим образом:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

Переменная DATABASES содержит набор конфигураций подключений к базам данных в виде словаря. Ключи в этом словаре - названия подключений. То есть мы можем определить кучу подключений. Но как минимум одно подключение должно быть определено в переменной DATABASES - подключение с именем default, которое представляет подключение по умолчанию.

Конфигурация каждого подключения может состоять из ряда параметров. По умолчанию указываются только два параметра. Параметр ENGINE указывает на используемый движок для доступа к БД. В данном случае это встроенный пакет django.db.backends.sqlite3.

Второй параметр - NAME указывает на путь к базе данных. По умолчанию база данных называется db.sqlite3. Для установки пути используется каталог из переменной BASE_DIR, которая задана в начале файла:

```
BASE_DIR = Path(__file__).resolve().parent.parent
```

По умолчанию BASE_DIR указывает на каталог, в котором находится папка проекта. И после первого запуска проекта в указанном каталоге по умолчанию будет создан файл db.sqlite3, который, собственно, и будет использоваться в качестве базы данных (рисунок 36).

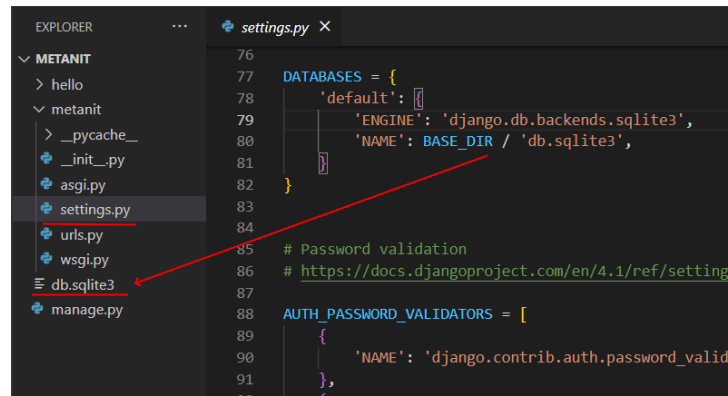


Рисунок 36 - Установка пути к базе данных в проекте Django

Чтобы использовать другие системы управления базами данных, необходимо будет установить соответствующий пакет.

Это встроенные движки, но также сообщество может предоставлять свои движки к другим СУБД. Например, компания Microsoft предоставляет пакет `mssql-django` для работы с MS SQL Server (устанавливается командой `pip install mssql-django`)

Выше в настройках по умолчанию применялись только два параметра: `ENGINE` и `NAME`. Однако при необходимости можно указать и дополнительные параметры:

- `ATOMIC_REQUESTS`: при значении `True` обортывает каждый запрос к бд в транзакцию. По умолчанию равно `False`;
- `AUTOCOMMIT`: при значении `False` отключает систему управления транзакциями Django. По умолчанию равно `True`;
- `ENGINE`: движок бд. Поддерживаются следующие встроенные движки:
 - `django.db.backends.postgresql`;
 - `django.db.backends.mysql`;
 - `django.db.backends.sqlite3`;
 - `django.db.backends.oracle`.
- `HOST`: хост базы данных;
- `NAME`: имя базы данных, для `sqlite` - путь к бд;
- `CONN_MAX_AGE`: время жизни подключения в секундах. Значение `0` закрывает подключение сразу после завершения каждого запроса к бд. А значение `None` указывает на постоянное подключение. По умолчанию равно `0`;
- `CONN_HEALTH_CHECKS`: при значении `True` проверяет активность подключения, и если оно закрыто, заново устанавливает соединение. По умолчанию равно `False`;
- `OPTIONS`: дополнительный набор параметров, который может потребоваться при подключении к определенным базам данным;
- `PASSWORD`: пароль. По умолчанию равно `"` (пустая строка);
- `PORT`: порт бд. По умолчанию равно `"`;
- `TIME_ZONE`: часовой пояс. По умолчанию равно `None`;

- USER: имя пользователя. По умолчанию равно ";
- TEST: набор настроек в виде словаря для теста подключения.

Для разных СУБД может потребоваться разный набор параметров. Например, подключение к БД Postgres в общем случае выглядит так:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'mydatabase',
        'USER': 'mydatabaseuser',
        'PASSWORD': 'mypassword',
        'HOST': '127.0.0.1',
        'PORT': '5432',
    }
}
```

При создании базы данных она еще пуста, не содержит никаких данных. Однако в консоли при запуске проекта мы можем увидеть некоторое сообщение, выделенное красным цветом (рисунок 37):

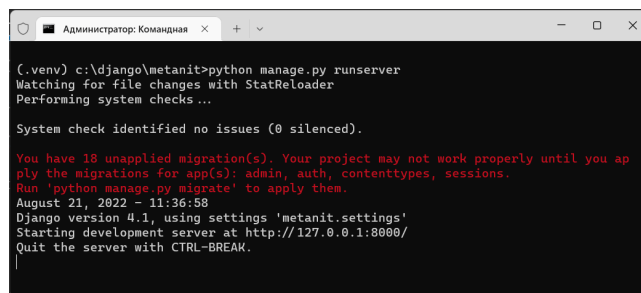


Рисунок 37 - Применение миграций базы данных для встроенных приложений в Django

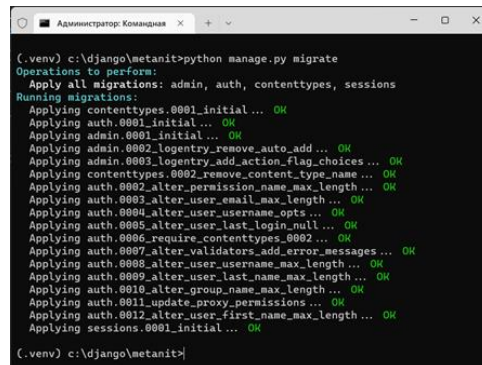
Дело в том, что для ряда приложений, подключенных в проект по умолчанию, требуется база данных для сохранения некоторых своих данных. Так, в файле settings.py в переменной INSTALLED_APPS по умолчанию указан ряд приложений

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    # остальные приложения .....
    # .....
]
```

Первые четыре приложения так или иначе используют базу данных. Однако, чтобы базы данных создала нужные для этих приложений таблицы,

необходимо выполнить миграции - специальные скрипты, которые изменяют структуру базы данных. Для этого выполним в консоли команду (рисунок 38):

```
python manage.py migrate
```



```
(.venv) c:\django\metanit>python manage.py migrate
Operations to perform:
Apply all migrations: admin, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial ... OK
  Applying auth.0001_initial ... OK
  Applying admin.0001_initial ... OK
  Applying admin.0002_logentry_remove_auto_add ... OK
  Applying admin.0003_logentry_add_action_flag_choices ... OK
  Applying contenttypes.0002_remove_content_type_name ... OK
  Applying auth.0002_alter_permission_name_max_length ... OK
  Applying auth.0003_alter_user_email_max_length ... OK
  Applying auth.0004_alter_user_username_opts ... OK
  Applying auth.0005_alter_user_last_login_null ... OK
  Applying auth.0006_require_contenttypes_0002 ... OK
  Applying auth.0007_alter_validators_add_error_messages ... OK
  Applying auth.0008_alter_user_username_max_length ... OK
  Applying auth.0009_alter_user_last_name_max_length ... OK
  Applying auth.0010_alter_group_name_max_length ... OK
  Applying auth.0011_update_proxy_permissions ... OK
  Applying auth.0012_alter_user_first_name_max_length ... OK
  Applying sessions.0001_initial ... OK
(.venv) c:\django\metanit>
```

Рисунок 38 - Миграции базы данных в Django

Если после этого мы откроем базу данных db.sqlite3, которая есть в проекте, в какой-нибудь специальной программе для просмотра БД SQLite, то мы увидим, что она содержит ряд таблиц (рисунок 39):

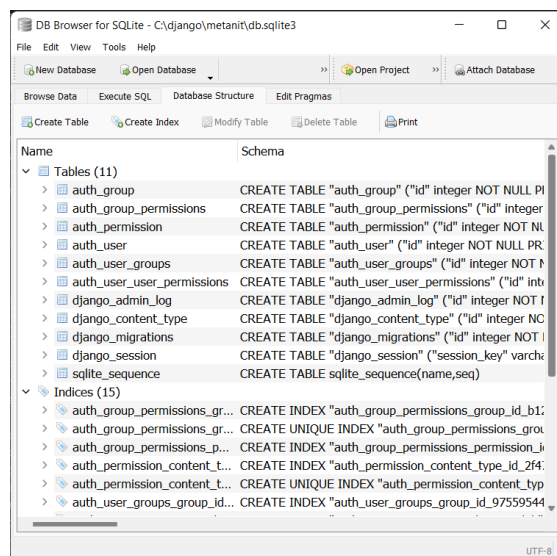


Рисунок 39 - Просмотр базы данных SQLite в Django

Для просмотра и работы с базами данных SQLite можно использовать бесплатную программу DB Browser for SQLite, которая доступна для всех основных операционных систем.

В частности, можно увидеть, что база данных содержит более десятка таблиц и индексов. Все они связаны с системой аутентификации и авторизации, администрирования и сессиями. И даже если мы непосредственно не будем использовать базу данных, но будем использовать системы аутентификации и авторизации, администрирования или сессии, то для нормальной работы нам все равно потребуется база данных и необходимо будет выполнить миграции.

Создание моделей

Модели в Django описывают структуру используемых данных. Используемые в программе данные хранятся в базах данных, и с помощью моделей как раз осуществляется взаимодействие с базой данных.

При создании приложения по умолчанию в его каталог добавляется файл `models.py`, который применяется для определения моделей. Модель представляет класс, унаследованный от `django.db.models.Model`.

Так, изменим файл `models.py` следующим образом (рисунок 40):

```
from django.db import models
class Person(models.Model):
    name = models.CharField(max_length=20)
    age = models.IntegerField()
```

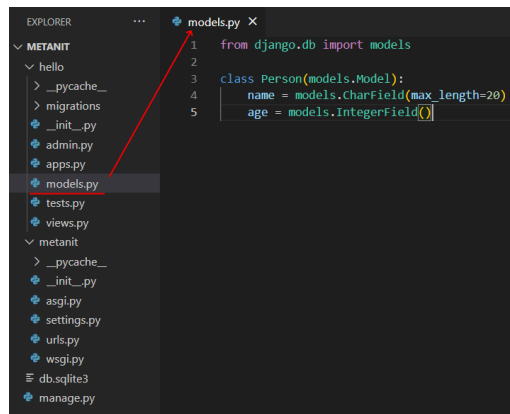
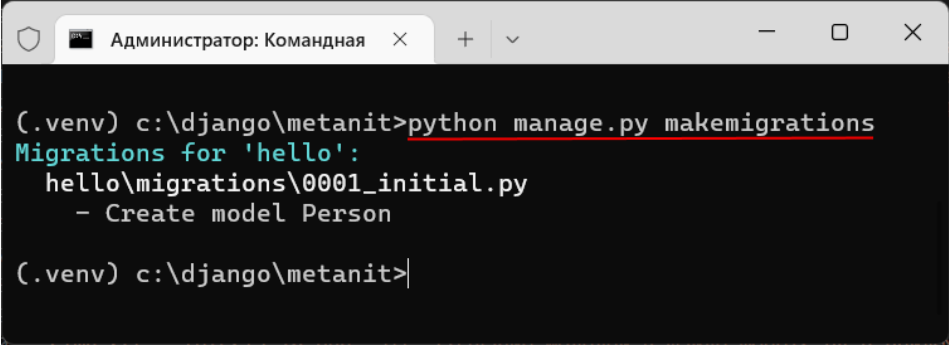


Рисунок 40 - Создание моделей в файле `models.py` в приложении на Django

Здесь определена простейшая модель, которая называется `Person` и которая представляет человека. В модели определены два поля. Поле `name` представляет тип `CharField` - текстовое поле, которое хранит последовательность символов. Оно будет хранить имя человека. Для `CharField` обязательно надо указать параметр `max_length`, который задает максимальную длину хранящейся строки. И поле `age` представляет тип `IntegerField` - числовое поле, которое хранит целые числа. Оно предназначено для хранения возраста человека.

Каждая модель сопоставляется с определенной таблицей в базе данных. Однако пока у нас нет в бд таблицы, которая хранит объекты модели `Person`. И в этом случае нам надо создать и выполнить миграцию. Миграция преобразует базу данных в соответствии с определением моделей.

Вначале необходимо создать миграцию с помощью команды (рисунок 41):
`python manage.py makemigrations`



```
(.venv) c:\django\metanit>python manage.py makemigrations
Migrations for 'hello':
  hello\migrations\0001_initial.py
    - Create model Person

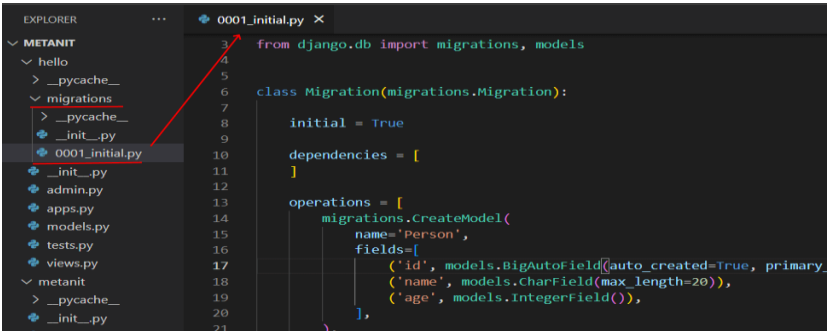
(.venv) c:\django\metanit>
```

Рисунок 41 - Создание миграции моделей в Django

После этого в приложении в папке migrations мы обнаружим новый файл, который будет иметь примерно следующее содержимое:

```
from django.db import migrations, models
class Migration(migrations.Migration):
    initial = True
    dependencies = [
    ]
    operations = [
        migrations.CreateModel(
            name='Person',
            fields=[
                ('id', models.BigAutoField(auto_created=True, primary_key=True,
serialize=False, verbose_name='ID')),
                ('name', models.CharField(max_length=20)),
                ('age', models.IntegerField()),
            ],
        ),
    ]
```

Это и есть миграция. Здесь можно заметить, что создается не два, а три поля - поле id, которое будет представлять первичный ключ, добавляется по умолчанию. Поэтому в принципе в самой модели нам не нужно явным образом определять какой-либо идентификатор (рисунок 42).



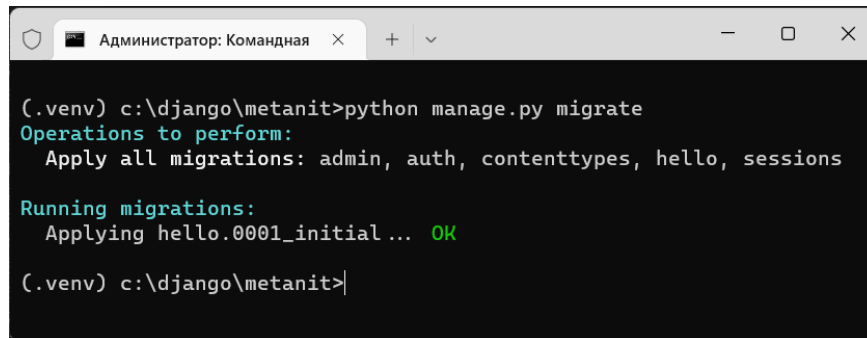
```
EXPLORER
METANIT
  hello
  > __pycache__
  migrations
  > __pycache__
  > __init__.py
  0001_initial.py
  > __init__.py
  admin.py
  apps.py
  models.py
  tests.py
  views.py
  metanit
  > __pycache__
  > __init__.py

0001_initial.py
1 from django.db import migrations, models
2
3
4
5
6 class Migration(migrations.Migration):
7
8     initial = True
9
10    dependencies = [
11    ]
12
13    operations = [
14        migrations.CreateModel(
15            name='Person',
16            fields=[
17                ('id', models.BigAutoField(auto_created=True, primary_key=True,
18serialize=False, verbose_name='ID')),
19                ('name', models.CharField(max_length=20)),
20                ('age', models.IntegerField()),
21            ],
22        ),
23    ]
```

Рисунок 42 - Миграция моделей в Django

Теперь надо выполнить данную миграцию. Для этого выполняется команда (рисунок 43):

`python manage.py migrate`



```
(.env) c:\django\metanit>python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, hello, sessions

Running migrations:
  Applying hello.0001_initial ... OK

(.env) c:\django\metanit>
```

Рисунок 43 - Выполнение миграции базы данных в Django

После этого, если мы откроем базу данных `db.sqlite3`, которая есть в проекте, то мы увидим, что в нее добавлена таблица для хранения данных модели `Person` (рисунок 44):

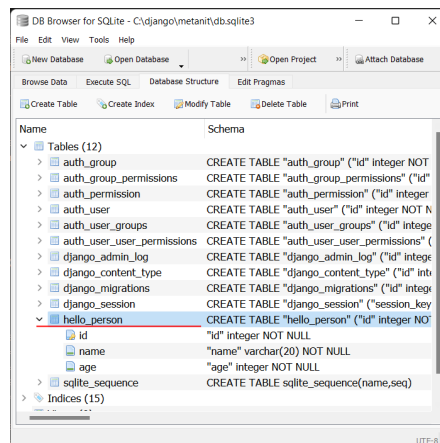


Рисунок 44- Базы данных SQLite в Django

Стоит учитывать, что в процессе работы над проектом определение модели может меняться - к ней могут добавляться новые поля, либо могут удаляться уже существующие, либо могут добавляться новые модели, данные которых мы также захотим хранить в базе данных. И в этом случае опять надо создавать миграцию и применять ее к базе данных, чтобы база данных была синхронизирована с определением моделей.

Практическая часть

Для выполнения практической части практической работы, Вам необходимо развернуть Django-проект. Вспоминая, 20 лабораторную работу самостоятельно разверните проект и отобразите на сайте текст «Hello, world!».

Подсказка по развертыванию проекта:

- `python -m venv venv;`
- `pip install django;`
- `django-admin startproject [имя проекта];`
- `./manage.py runserver` (или `python manage.py runserver`);
- `./manage.py migrate` (или `python manage.py migrate`);
- `django-admin startapp [имя приложения];`
- `INSTALLED_APPS =>` Добавить 'index';
- создать `templates =>` Создать 'index.html' (В тегах первого уровня добавить текст «Hello, world!»);
- `DIRS =>` Добавить 'templates';
- создать представление;
- добавить шаблон для URL-адресов (path).

Для добавления какого-либо функционала сайта, почти всегда применяют базу данных, но для того, чтобы мы не морочили себе голову с языком SQL, в Django все реализовано с помощью языка Python, прямо внутри кода. Базу данных надо воспринимать как хранилище из таблиц.

Создать модель в БД = Создать таблицу в БД. Представим, что нам нужно реализовать работу с сотрудниками на сайте, т.е. нам необходимо создать список сотрудников. Для этого, зайдём в файл моделей приложения `index (models.py)`. И пропишем код (рисунок 45):

```
1  from django.db import models
2
3  class Worker(models.Model):
4      pass
```

Рисунок 45 - Пустая модель сотрудника (Worker)

Что здесь происходит? Мы создаем класс `Worker` (Сотрудник, работник), который наследуется от `models.Model` и также, ещё стоит заглушка `pass`, означающая то, что код ничего не будет делать.

Определим поля для нашей таблицы сотрудников, такие как имя, фамилия, зарплата (рисунок 46).

```
1  from django.db import models
2
3  class Worker(models.Model):
4      name = models.CharField(max_length=25, blank=False)
5      surname = models.CharField(max_length=50, blank=False)
6      salary = models.IntegerField(default=0)
```

Рисунок 46 - Заполненная модель сотрудника (Worker)

Что здесь происходит? Создаем переменные, которые будут иметь свой определенный тип данных.

Тип данных в программировании — классификация различных значений,

которые могут использоваться в программах.

CharField – для работы со строковой информацией. *max_length* – максимальная длина строки не будет превышать n-символов и поле НЕ МОЖЕТ быть пустым.

IntegerField – для работы с числами. *default* – для установки значения по умолчанию (установим значение по умолчанию 0)

Готово! Но, Django не знает о нашей только что созданной модели – ничего!

Для того, чтобы модель появилась в нашем проекте, нужно провести миграции (рисунок 47).

```
PS D:\django-learning\hello\learning_django> python manage.py makemigrations
Migrations for 'index':
  index\migrations\0001_initial.py
  - Create model Worker
```

Рисунок 47 - Проведение миграций

И далее, необходимо выполнить проведенные миграции командой `python manage.py migrate` (рисунок 48).

```
PS D:\django-learning\hello\learning_django> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, index, sessions
Running migrations:
  Applying index.0001_initial... OK
```

Рисунок 48 - Выполнение миграций

Готово! Модель в БД создана!

Как создавать записи в нашей таблице сотрудников?

Необходимо зайти в админ-панель Django-проекта. Для этого, измените поисковую строку (рисунок 49):



Рисунок 49- URL-адрес административной панели Django

К сожалению, или к счастью, без логина и пароля никто не разрешит нам войти в админ-панель нашего приложения. Для создания админа (или же – суперпользователя), перейдем в терминал и пропишем команду (рисунок 50):

```
PS D:\django-learning\hello\learning_django> python manage.py createsuperuser
Username: █
```

Рисунок 50 - Создание суперпользователя

Суперпользователь в Django — главный пользователь сайта с максимальными правами.

Придумываем имя для нашего админа, указываем почту (необязательно) и пароль.

При написании пароля, введенные нами символы в консоли отображаться НЕ БУДУТ!

Далее, если пароль будет не надежным, Django предупредит нас об этом, проигнорируем его, т.к. мы работаем на локальном сервере, и при необходимости мы сможем поменять пароль, если будет публиковать приложение на сервер, и отправляем в терминал символ у (yes) (рисунок 51).

```
PS D:\django-learning\hello\learning_django> python manage.py createsuperuser
Username: gazik
Email address:
Password:
Password (again):
The password is too similar to the username.
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.
```

Рисунок 51 - Успешное создание суперпользователя

Войдем в админ-панель, используя только что созданные данные для входа (рисунок 52).

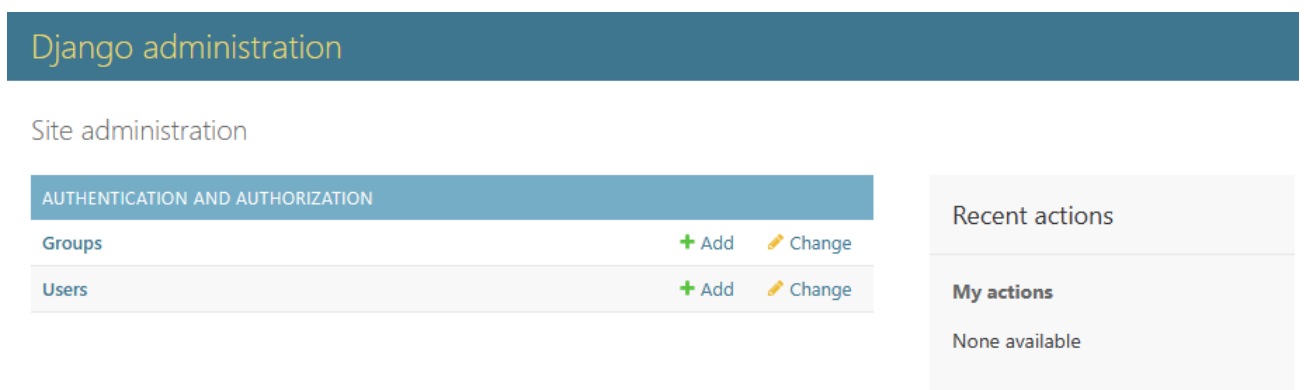


Рисунок 52 - Вид административной панели Django

Войдя в админ-панель, мы видим, автоматически созданную модель пользователей (User) (здесь будет отображаться список всех пользователей, которые зарегистрируются на сайте).

Зарегистрируем нашу модель в файле admin.py (рисунок 53).

```

1  from django.contrib import admin
2  from .models import Worker
3
4  admin.site.register(Worker)

```

Рисунок 53- Регистрация модели в файле admin.py

Что здесь происходит? Мы импортируем из файла моделей модель сотрудника и регистрируем на сайте модель. Все очень просто (рисунок 54)!

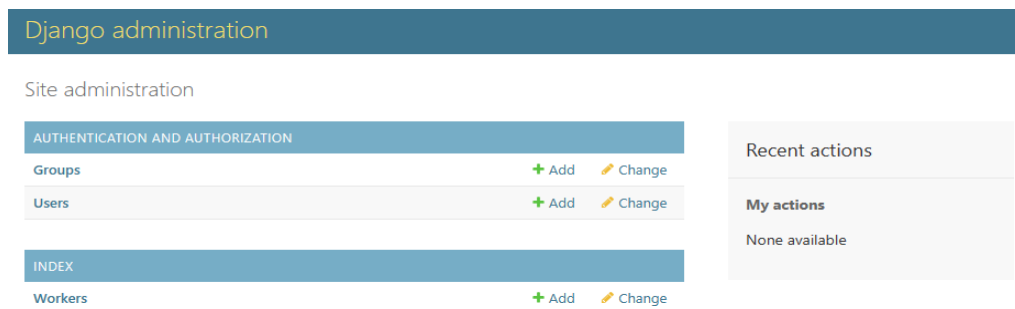


Рисунок 54- Зарегистрированная модель Workers в админ-панели

Добавим сотрудника в БД. Нажмем на Add и потом, напомним имя, фамилию и зарплату (рисунок 55).

 The screenshot shows the 'Add worker' form in the Django administration interface. The breadcrumb trail at the top reads 'Home > Index > Workers > Add worker'. On the left, there is a sidebar with a search bar and a list of links: 'Groups' and 'Users' under 'AUTHENTICATION AND AUTHORIZATION', and 'Workers' under 'INDEX'. The 'Workers' link is highlighted in yellow. The main form area is titled 'Add worker' and contains three input fields: 'Name' with the value 'Иван', 'Surname' with the value 'Иванов', and 'Salary' with the value '50000'. At the bottom of the form are three buttons: 'SAVE', 'Save and add another', and 'Save and continue editing'.

Рисунок 55 - Добавление сотрудника

Добавим ещё двух сотрудников, и что мы видим (рисунок 56)?

Select worker to change

Action: 0 of 3 selected

☐	WORKER
☐	Worker object (3)
☐	Worker object (2)
☐	Worker object (1)

3 workers

Рисунок 56 - Worker object (n)

Мы не можем понять, кто есть кто, исправим данный недочет. Зайдем в модели, и добавим код для магического метода `str`. Он отображает информацию об объекте класса для пользователей (рисунок 57).

```

1  from django.db import models
2
3  class Worker(models.Model):
4      name = models.CharField(max_length=25, blank=False)
5      surname = models.CharField(max_length=50, blank=False)
6      salary = models.IntegerField(default=0)
7
8      def __str__(self) -> str:
9          return f'{self.surname} {self.name}'
10

```

Рисунок 57 - Строковое представление для модели Worker

Теперь, наши сотрудники будут отображаться корректно. У них будет фамилия и имя (рисунок 58).

Django administration

Home > Index > Workers

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

Groups [+ Add](#)

Users [+ Add](#)

INDEX

Workers [+ Add](#)

Select worker to change

Action: 0 of 3 selected

☐	WORKER
☐	Петров Арсений
☐	Газизуллин Данил
☐	Иванов Иван

3 workers

Рисунок 58 - Корректное отображение данных о сотрудниках в админ-панели

Ну согласитесь, так намного лучше!

Зайдем в файл представлений и пропишем код (рисунок 24):

```
1 from django.shortcuts import render
2 from .models import Worker
3
4 def index_page(request):
5     all_workers = Worker.objects.all()
6     print(all_workers)
7
8     return render(request, 'index.html')
```

Рисунок 59 - Вывод всех сотрудников модели Worker в консоли

Что здесь происходит? Мы создаем переменную, где будет храниться список всех сотрудников и выводим его в терминал (рисунок 60).

```
<QuerySet [<Worker: Иванов Иван>, <Worker: Газизуллин Данил>, <Worker: Петров Арсений>]>
[23/Jan/2024 17:46:38] "GET / HTTP/1.1" 200 233
```

Рисунок 60 - Вывод в консоли всех сотрудников модели Worker

А что, если нужно получить записи по некому фильтру! (рисунок 61)?

```
1 from django.shortcuts import render
2 from .models import Worker
3
4 def index_page(request):
5     worker_salary = Worker.objects.filter(salary=70000)
6     print(worker_salary)
7     return render(request, 'index.html')
```

Рисунок 61 - Вывод данных по фильтру зарплаты в 70000

Мы получим результат в виде вывода данных в консоли с фильтром в виде зарплаты в 70000 (рисунок 62):

```
<QuerySet [<Worker: Газизуллин Данил>]>
[23/Jan/2024 17:51:27] "GET / HTTP/1.1" 200 233
```

Рисунок 62 - Отфильтрованный вывод данных в консоли

Красивый вывод списка всех сотрудников в консоли (имя, фамилия, зарплата) (рисунок 63).

```

1 from django.shortcuts import render
2 from .models import Worker
3
4 def index_page(request):
5
6     all_workers = Worker.objects.all()
7
8     for worker in all_workers:
9         print(worker.id, worker.name, worker.surname, worker.salary)
10
11
12     return render(request, 'index.html')

```

Рисунок 63 - Вывод списка всех сотрудников в консоли через цикл for

Если посмотреть в консоль, то мы увидим следующий вывод (рисунок 64):

```

1 Иван Иванов 50000
2 Данил Газизуллин 70000
3 Арсений Петров 105000
[23/Jan/2024 19:53:46] "GET / HTTP/1.1" 200 233

```

Рисунок 64 - Вывод данных в консоли через цикл for

Как добавить новую запись? По простым принципам ООП. Советую изучить работу с ООП и знать её хотя бы на базовом уровне (рисунок 65).

```

1 from django.shortcuts import render
2 from .models import Worker
3
4 def index_page(request):
5
6     all_workers = Worker.objects.all()
7
8     leila_gatufovna = Worker(name='Лейла', surname='Галимова', salary='200000')
9     leila_gatufovna.save()
10
11     return render(request, 'index.html')

```

Рисунок 65 - Добавление нового сотрудника

Как изменить запись? Все очень просто (рисунок 66):

```

1 from django.shortcuts import render
2 from .models import Worker
3
4 def index_page(request):
5
6     all_workers = Worker.objects.all()
7
8     changed_worker = Worker.objects.get(id=1)
9     changed_worker.surname = 'Дубров'
10    changed_worker.save()
11
12    return render(request, 'index.html')

```

Рисунок 66 - Редактирование сотрудника

Если посмотреть таблицу сотрудников, то можно заметить, что «Иван Иванов» стал «Иван Дубров» (рисунок 67).

Select worker to change

Action: Go 0 of 4 selected

☐	WORKER
☐	Галимова Лейла
☐	Петров Арсений
☐	Газизуллин Данил
☐	Дубров Иван

4 workers

Рисунок 67 - Измененные данные сотрудника (фамилия)

Много кода, не так ли? Воспользуемся специальным методом update (рисунок 33).

```
1 from django.shortcuts import render
2 from .models import Worker
3
4 def index_page(request):
5
6     Worker.objects.filter(id=1).update(name='Дмитрий')
7
8     return render(request, 'index.html')
```

Рисунок 68 - Использование метода update для редактирования данных

И мы также увидим изменение в данных сотрудника (рисунок 34):

Select worker to change

Action: Go 0 of 4 selected

☐	WORKER
☐	Галимова Лейла
☐	Петров Арсений
☐	Газизуллин Данил
☐	Дубров Дмитрий

4 workers

Рисунок 69 - Измененные данные сотрудника (имя)

Как удалить запись в таблице? Не прижился, ты Иван (Дмитрий) Иванов (Дубров), пора бы тебя и удалить. Для этого воспользуемся фильтром по id и методом delete (рисунок 35).

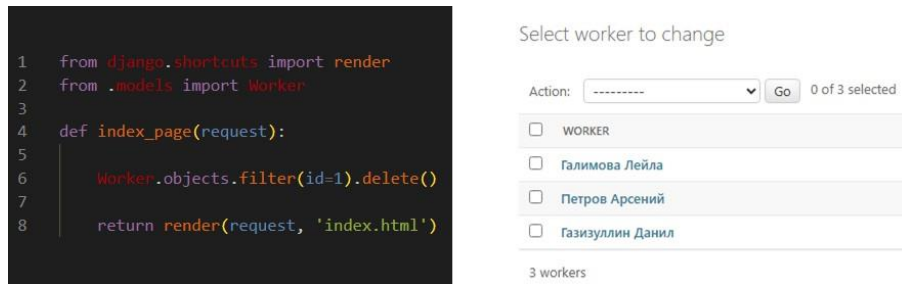


Рисунок 70 - Использование фильтра и метода delete для удаления данных из таблицы Worker

Пример решения задачи на языке программирования Python

Задача. В ранее созданном проекте `blog_site` и приложении `posts` создайте модель `Post`, предназначенную для хранения публикаций блога. Модель должна содержать следующие поля: заголовок публикации, текст публикации, дата создания и автор. После создания модели выполните миграции базы данных. Затем зарегистрируйте модель в административной панели Django в файле `admin.py`. Создайте администратора системы и войдите в административную панель по адресу `http://127.0.0.1:8000/admin`. Добавьте через административную панель не менее пяти публикаций блога.

Решение задачи:

1. Создание модели (`models.py`)

```

from django.db import models
class Post(models.Model):
    title = models.CharField(max_length=255)
    content = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)
    author = models.CharField(max_length=100)
    def __str__(self):
        return self.title

```

2. Выполнение миграций

```

python manage.py makemigrations
python manage.py migrate

```

3. Регистрация модели в админке (`admin.py`)

```

from django.contrib import admin
from .models import Post
admin.site.register(Post)

```

4. Создание администратора

```

python manage.py createsuperuser

```

5. Запуск сервера

```

python manage.py runserver

```

6. Проверка работы

Откройте в браузере:

`http://127.0.0.1:8000/admin`